

REFERENCE GUIDE · 2026 EDITION

AI Agent 开发参考手册

MCP · 多 Agent · 提示词工程 · 自进化架构 · 基准数据

97M

月 SDK 下载

10K+

MCP Servers

80.8%

Claude SWE-bench 知识章节

基于最新网络研究与 Anthropic 官方文档 · 更新：2026-03

现状数据（2026 - 03）

MCP 于 2024 - 11 由 Anthropic 发布，2025 - 12 捐赠 Linux 基金会。1000 万月度 SDK 下载、1,200+ 认证实现。OpenAI 和 Google DeepMind 均支持 MCP。

三种传输方式

方式	参数	适用场景	推荐度
HTTP	--transport http	远程服务、生产环境	★★★ 首选
SSE	--transport sse	流式数据、实时推送	★★☆ 次选
Stdio	--transport stdio	本地进程、系统访问	★★☆ 本地

配置文件三级作用域

文件	作用域	提交 git
.mcp.json	项目级，团队共享	应提交
.claude/settings.local	项目级，本地私有	不提交
~/ .claude/settings.local	全局，跨项目	— 个人配置

快速配置示例

```
py # HTTP 连接 Claude MCP
py claude mcp add --transport http notion https://mcp.notion.com
py
py # SSE 连接 Claude MCP
py claude mcp add --transport sse asana https://mcp.asana.com
py
py # Stdio 连接 Claude MCP
py claude mcp add --transport stdio --env AIRTABLE_API_KEY=your-key
py -- npx -y airtable-mcp-server
```

最小必装三件套

GitHub + Brave Search + Playwright 覆盖 90% 的开发日常需求。

模式	描述	适用场景
顺序链	$A \rightarrow B \rightarrow C$ ，前者输出作为后者输入	数据流水线、文档生成
并行执行	多 Agent 同时处理不同子任务	大规模分析、独立模块开发
主管路由	Orchestrator 决定调用哪个子 Agent	客服路由、多领域专家系统
层级嵌套	多层 Orchestrator 嵌套	复杂企业流程、研究系统

最常见反模式

把所有能力（角色 + 规则 + 工具说明 + 领域知识）塞进一个超长 System Prompt → 10,000+ tokens
每次消耗，注意力稀释，脆弱难维护。

正确架构：两层分离

层级	文件	内容	加载时机
身份层	AGENTS.md	角色、行为规则、约束策略	始终加载
能力层	SKILLS.md 文件集	领域专项能力、工具调用方式	按需动态加载

渐进式披露 (Progressive Disclosure)

```
Level 1: Claude Skill v0.1
! R$e-f/T&N _SRM÷lBvøQs
! b g,ÿ kÎN* Skill NÂQàN* tokens

Level 2: R·}-6kµÿ SêR·}S9'Mv,,ÿ
R·}[Æet SKILL.md <æ~Æc Nä
! S T+ke$ð0 y:0<0 ]âQw'M·n
! N S9'Mv,, Skill l8•ÜN m^€ N N e†
```

核心收益

收益	说明
上下文效率	只有相关技能被加载，保留宝贵上下文窗口
可复用性	同一 Skill 可被多个 Agent 共享（如代码审查 Skill）
可维护性	修改某个 Skill 不影响 Agent 身份，变更隔离
可扩展性	添加新 Skill 无需重建 Agent，容量线性增长

⚠ 供应商锁定警告

Claude Agent Skills 标准（2025-12 发布）是 Anthropic 专有标准。无法直接迁移到 GPT 或 Gemini。生产系统请考虑抽象层。

范式转变：提示词工程 → 上下文工程

维度	提示词工程（旧）	上下文工程（新）
关注点	如何措辞	在上下文中放什么信息
目标	让模型输出正确格式	用最少的 tokens 最大化期望输出概率
层级	单次交互优化	整个 Agent 系统的信息架构

Andrej Karpathy 的定义（2025）

"上下文工程是一门微妙的艺术与科学——用恰好合适的信息填充上下文窗口，以最大化下一步的期望输出概率。"

针对不同模型类型的策略

```
pŷc"t W< j!W< ŷ Claude Extended Thinking , o 1ŷ ŷ
pŷ !' ~ÜšØ\Bvîh ŷ OâNûj!W<•ê^LR %ã~Æ,,
pŷ !' |{kôŷ Tj<É•Dmñ]âz ^ "%•%ãQ³NÀNH "
pŷ O<ŷ Ø [žs°šØ^vSÑm^`o- R ŷ € †QT yÍ•¹• `ÅQµØ
pŷ
pŷc Nâ•u_aW< j!W< ŷ Claude Haiku , GPT - 4 oŷ ŷ
pŷ !' ~Üf xnv,,R kec Nâ
pŷ !' |{kôŷ ~ÜR ~š]âz ^ Q™<æ~ÆNÜRin SU
pŷ O<ŷ Ø 1 .R ^úe†Nö 2 .mÜR |{ 3 .[žs°e¹lŌ 4 .Q™mK<ŌØ
```

R e A c t 框架（思考 - 行动 - 观察循环）

```
pŷ Thought : [R g•_SRMrŷ` ŷ xn[šN N ke ]
pŷ Action : []âQwT yð ]
pŷ Action Input : { "SÂep " : "P< " }
pŷ
pŷe6R0~ŌgæT ŷ
pŷ Observation : []âQw•ŌVPv,,Q...[¹ ]
pŷ Thought : [WÚNŽ~Ōgæv,,e°R g• ]
pŷ . . ŷ _as™•ŌNÜRi[Œb ŷ
pŷ
pŷ Final Answer : [g ~È{ThH ]
```

实用上下文优化对比

场景	低效做法	高效做法
代码库分析	一次性加载所有文件	动态加载相关文件（RAG）
长对话	保留全部历史	定期压缩，保留关键决策节点
工具文档	全部工具说明放 Prompt	技能路由，只加载相关工具
多轮调用	每次重发完整上下文	利用缓存，相同前缀 90% 折扣

SOLID 原则映射

原则	Agent 应用
S 单一职责	一个 Agent 一个领域，不做 "全能 Agent"
O 开闭原则	通过新增 Skill 扩展能力，不修改 Agent 核心身份
L 里氏替换	专化 Agent 可替换通用 Agent 而不破坏系统
I 接口隔离	工具接口精确匹配 Agent 需求，不暴露无关接口
D 依赖倒置	Agent 依赖工具抽象（MCP），不依赖具体实现

六层知识管理架构

Layer 6 - Working Memory	Context Window
Layer 5 - Retrieval	Agentic RAG
Layer 4 - Procedural	SKILL.md Library
Layer 3 - Episodic	Mem0 / Session Store
Layer 2 - Graph	Neo4j / AWS Neptune
Layer 1 - Semantic	PostgreSQL + pgvector

PostgreSQL + pgvector 成为 2025 年 GenAI 标准

三笔大额收购证明市场方向：Snowflake 收购 Crunchy Data (2.5 亿\$)、Databricks (10 亿\$)、Supabase E 轮 (1 亿\$)。pgvector 让 PostgreSQL 直接支持向量数据库。

重要警告

当前 LLM 对抽象软件工程原则的理解有限。不要依赖模型自动应用 SOLID 原则——架构师必须在系统设计阶段手动强制执行。

进化设计三个维度

维度	选项
进化什么	模型参数、提示词、记忆、工具集、工作流图、Agent 群体角色
何时进化	任务内（测试时反思、在线RL）或跨任务（跨episode微调）
如何进化	梯度RL、模仿学习、种群进化算法、元学习

四大核心框架

框架	核心机制	关键特点
Darwin Gödel Machine	维护代码库版本树	只有实证超越先祖才被接受，可量化安全性
RoboPhD (2026-01-11)	实例化→跨污染→ELO锦标赛→错误A	产生前所未见的涌现能力
GEPA (OpenAI)	采样轨迹→反思→提议修订→帕累托评估	可直接实现的提示词自动优化
EvoAgentX (EMNLP'25)	自动化(5A)gentic 工作流进化	EMNLP'25 展示，开源可用

GEPA 核心循环（可直接使用）

```
py def gepa_loop(agent, task_samples, iterations=10):
py     current_prompt = agent.system_prompt
py     for i in range(iterations):
py         # 1. 'Çh7bg^L·h·ù
py         trajectories = [agent.run(t) for t in task_samples]
py         # 2. •êq6<íŠ Sí` R g·Y1·%SŸVà
py         reflection = reflect_on_trajectories(trajectories)
py         # 3. cĐ<@0î<¢
py         revised = propose_revision(current_prompt, reflection)
py         # 4. ^ }/bX<Ä00ÿ Sê0ŸuYN • kev„0î<¢ÿ
py         if is_pareto_better(eval(revised), eval(current_prompt)):
py             current_prompt = revised
py     return current_prompt
```

自进化系统强制安全要求

- ① 必须有版本回退机制（进化失败时能回退到已知良好状态）
- ② 代码 / 模型更新前必须实证证明性能提升
- ③ 所有进化实验必须在沙箱环境中执行
- ④ 设置性能下限阈值，低于阈值自动拒绝变更。

编码能力基准（SWE - bench , 2026 - 03）

模型	上下文	SWE - bench	相对成本
Claude Opus 4.6	200K	80.8% ★	5x
Claude Sonnet 4.6	1M	79.6%	1x（推荐）
GPT-5 agent ic codi	128K	74.5%	—
Claude Opus 4.1	200K	74.5%	—

框架对比（2026 年生产级）

框架	最适合	主要缺点
LangGraph	生产级、合规、精细控制	学习曲线较高
CrewAI	快速原型、角色化 workflow	token 消耗 2x，速度 3x 慢
AutoGen / MS	代码生成、迭代修改	控制粒度较低
OpenAI Agents	与 OpenAI 原生、低延迟	生态锁定
LlamaIndex	文档 / 知识库问答	通用 Agent 能力弱

关键事件时间线

- py2024-11 Anthropic 发布 MCP ("AI 的 USB-C")
- py2025-03 OpenAI Agents SDK 发布；Manus AI 发布，GAIA 三级全 SOTA
- py2025-10 Microsoft Agent Framework (AutoGen + Semantic Kernel)
- py2025-11 Claude 高级工具使用（工具搜索 / 程序化调用 / 工具使用示例）
- py2025-12 Agent Skills 发布；MCP 捐赠 Linux 基金会；Meta 收购 Manus (AI 框架)
- py2026-02 16 个 Opus 4.6 Agent 用 Rust 写出能编译 Linux 内核的 C 编译器
- py2026-03 本文档发布；Sonnet 4.6：79.6%，Opus 4.6：80.8% SWE-bench

框架选型建议

需要精细控制→LangGraph；快速原型→CrewAI；代码生成→AutoGen；RAG密集→LlamaIndex；深度Claude集成→Agent SDK；不写代码→Dify。